Defining Security with Games

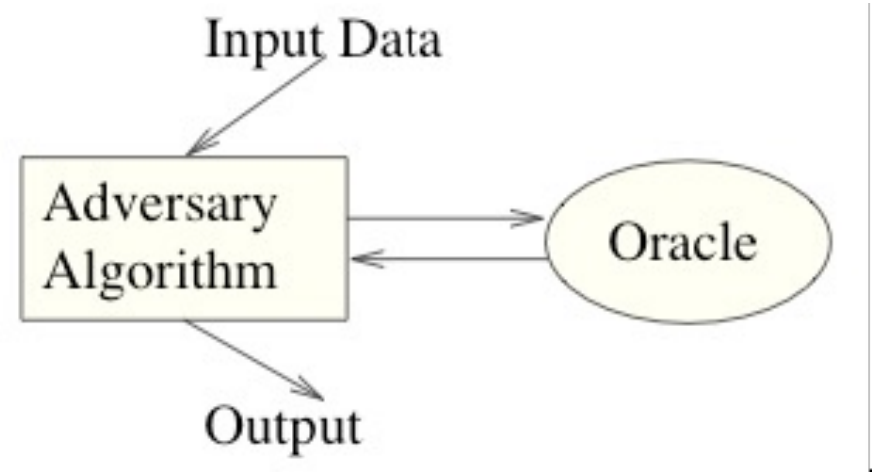
No perfect security – many cryptographic functions are not unconditionally secure

- In theory most cryptographic functions can be broken
- A realistic adversary with bounded computing power will break a secure function with only very small probability
- Nevertheless we want to argue formally about the security!

Games are frequently used to model formal security properties!

- Cryptosystem is considered secure if no adversary can win the game with significantly greater probability than an adversary who just guesses randomly

Model of Attack



We can think of the adversary as playing a game:

- **Input: Whatever adversary necessarily knows from the beginning, e.g., public key, distribution of plain texts, etc.**
- **Oracle: Models information adversary can obtain during an attack. Different kinds of information characterise different types of attacks.**
- **Output: Whatever the adversary wants to compute, e.g., secret key, partial information on plain text, etc. He wins if he succeeds.**

Attacks

1. Cyphertext Attack
   Given: $eK(x_1), eK(x_2)$ …
   Goal: deduce x_1, x_2 ,…, or K

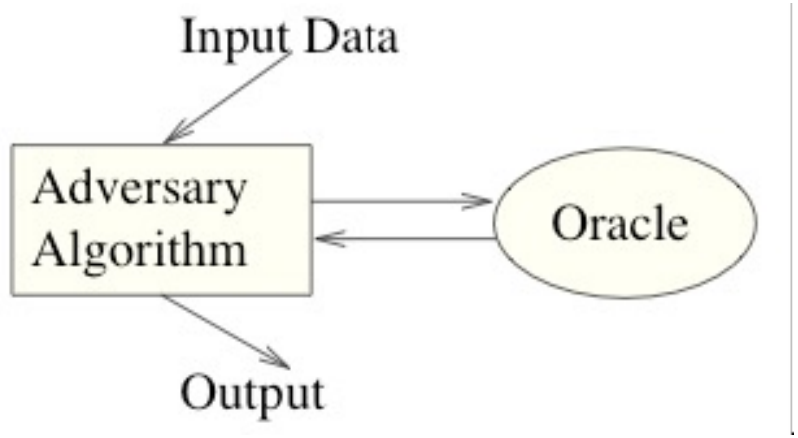
2. Known Plaintext Attack
   Given: $(x_1, eK(x_1)), x_2, eK(x_2)), \ldots$
   Goal: deduce K

3. Chosen Plaintext Attack
   like 2, but the attacker can choose x_i

4. Adaptive Chosen Plaintext Attack
   can not only choose plaintext, but can modify the plaintext based on encryption results

5. Chosen ciphertext
   Attacker can chose different ciphertexts to be decrypted and gets access to the decrypted plaintext.

Cryptographic Security



1. **Specify an oracle (a type of attack).**
2. **Define what the adversary needs to do to win the game, i.e., a condition on his output.**
3. **The system is secure under this definition, if any efficient adversary wins the game with only negligible probability.**

Example: Conventional Encryption

No input data for adversary.

Choose plaintext attack of following kind:

- Case 1: when asked to encrypt message *x*, oracle returns encryption of *x* under a fixed key that is randomly chosen initially;

  or:

- Case 2: oracle returns encryption of a randomly chosen message, totally independent of *x*.

Idea:

- In case 2, adversary gets completely useless data.
- If he cannot tell this apart from correct encryptions, he cannot do any damage in the real world (case 1) either.

Definitions

A, the alphabet, is a finite set

$M \subseteq A^*$ is the message space. $x \subseteq M$ is a plaintext (message)

C is the ciphertext space, whose alphabet may differ from M

K denotes the key space of keys

Each $K \in K$ determines a bijective function from M to C, denoted by $eK(x)$; this is the encryption function

For each $K \in K$, $dK(x)$ denotes a bijection from C to M; this is the decryption function.

Applying $eK(x)$ (or $dK(x)$) is called encryption (or decryption)

Encryption/decryption

An encryption scheme (or cipher) consists of a set $\{ eK_1 : K_1 \in K \}$ and a corresponding set $\{ dK_2 : K_2 \in K \}$ with the property that

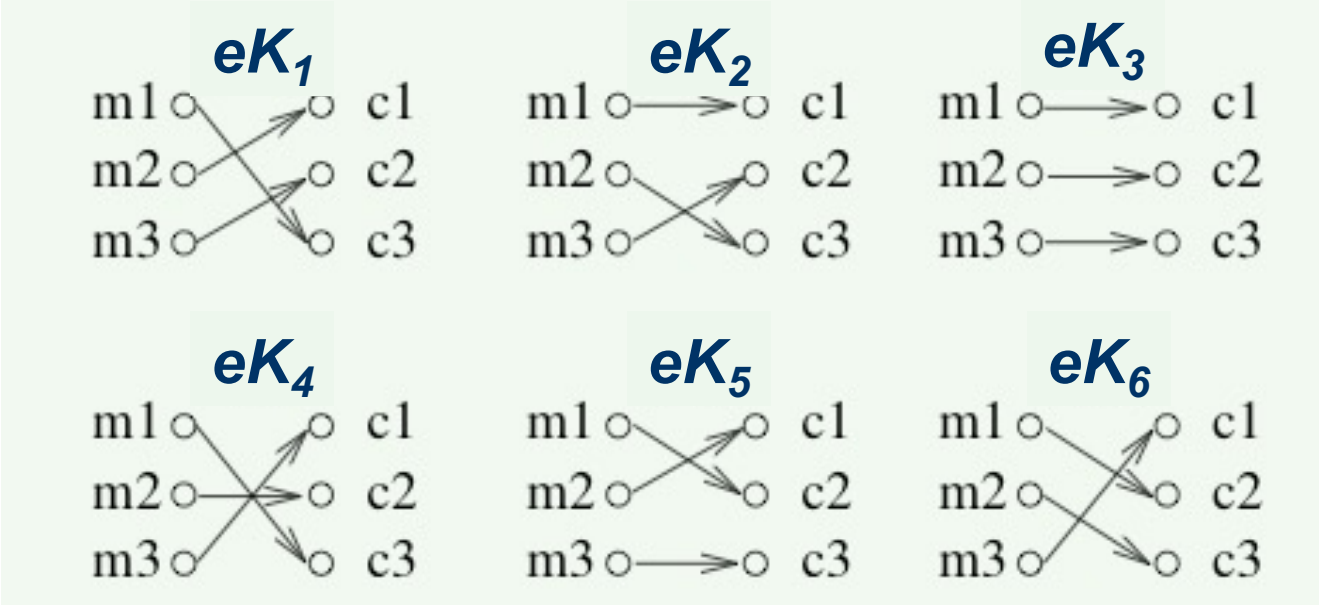
- for each $K_1 \in K$ there is a unique $K_2 \in K$, such that $dK_2(eK_1(x)) = x$, for all $x \subseteq M$

K_2 and K_1 form a key pair

To construct an encryption scheme requires fixing a message space M, a ciphertext space C, and a key space K, as well as encryption transformations $\{ eK_1 : K_1 \in K \}$ and corresponding decryption transformations $\{ dK_2 : K_2 \in K \}$

Example

Let $\mathrm{M} = \{m_1, m_2, m_3\}$ and $\mathrm{C} = \{c_1, c_2, c_3\}$. There are 3! = 6 bijections from M to C. The key space $\mathrm{K} = \{K_1, K_2, K_3, K_4, K_5, K_6\}$ specifies these transformations:



Suppose Alice and Bob agree on the transformation eK_1. To encrypt m_1, Alice computes $eK_1(m_1) = c_3$. Bob decrypts c_3 by reversing the arrows on the diagram and observing that c_3 points to m_1.

Symmetric Key Cryptography

An encryption scheme $(\{eK_1 : K_1 \in K\}, \{ dK_2 : K_2 \in K\})$ is symmetric if for each pair (eK_1, dK_2) it is computationally "easy" to determine dK_2 knowing only eK_1 (and vice versa).

In practice: $eK_1 = dK_2$

Symmetric ciphers (secret key cryptography): same key is used for encryption & decryption

- Encryption protects e.g. documents on the way from A to B
- A and B have to share a key and keep their keys secret
- A procedure is required for A and B to obtain their shared key
- For *n* parties to communicate directly, about n^2 keys are needed (serious drawback in practice…)

Block Ciphers vs. Stream Ciphers

A **block cipher** is an encryption scheme that breaks up the plaintext message into strings (blocks) of a fixed length l and encrypts one block at a time.

A **stream cipher** is one where the block-length is 1

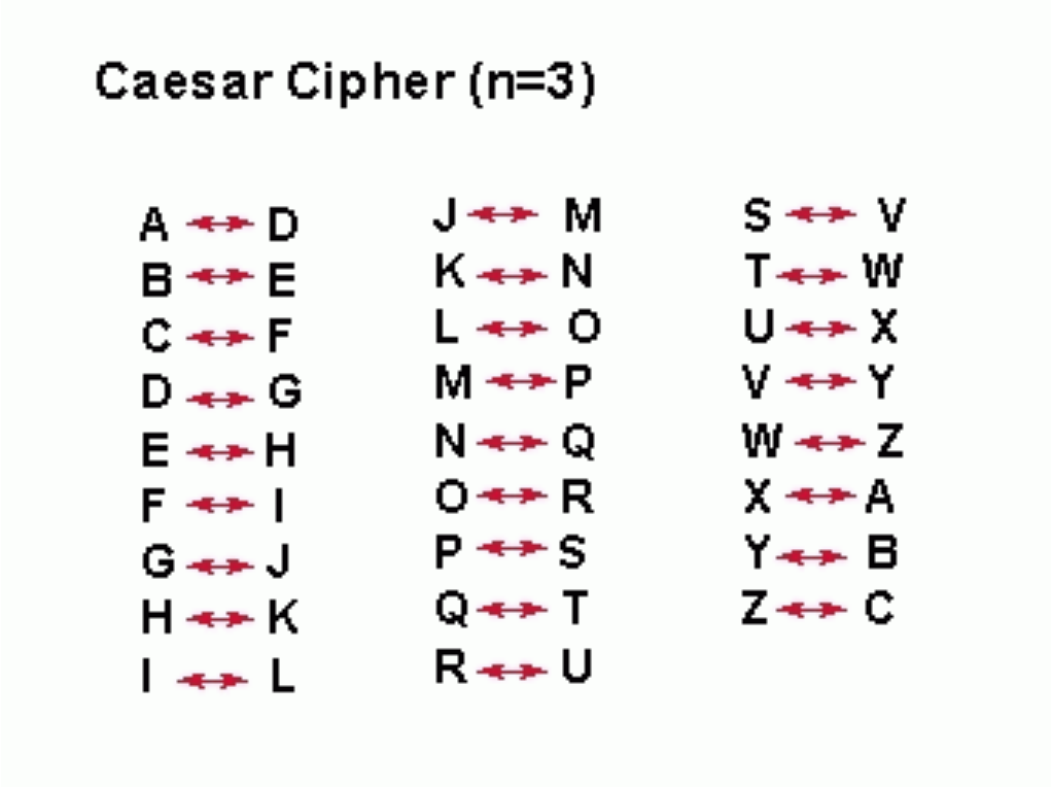
Codes work on words of varying length

- Translation is defined by a 'code-book', e.g.:

Word	*Code*
The	701
secret	5603
mischiefs	4008
that	3790
I	2879
set	0524
...	...

Caesar Cipher (n=3)

If we use the algorithm of simply moving each letter *n* places down the alphabet then the original alphabet we were using, or the plain text becomes a cipher text as follows:



Mono-alphabetic substitution ciphers

Let K be the set of all permutations on the alphabet A.

Define for each $K \in \mathcal{K}$ an encryption transformation eK on strings $x = x_1x_2...x_n \in \mathcal{M}$ as

$$eK(x) = K(x_1)K(x_2) \cdot \cdot \cdot K(x_n) = c_1c_2...c_n = c$$

To decrypt c, compute the inverse permutation

$$dK(c) = (eK(x))^{-1}$$

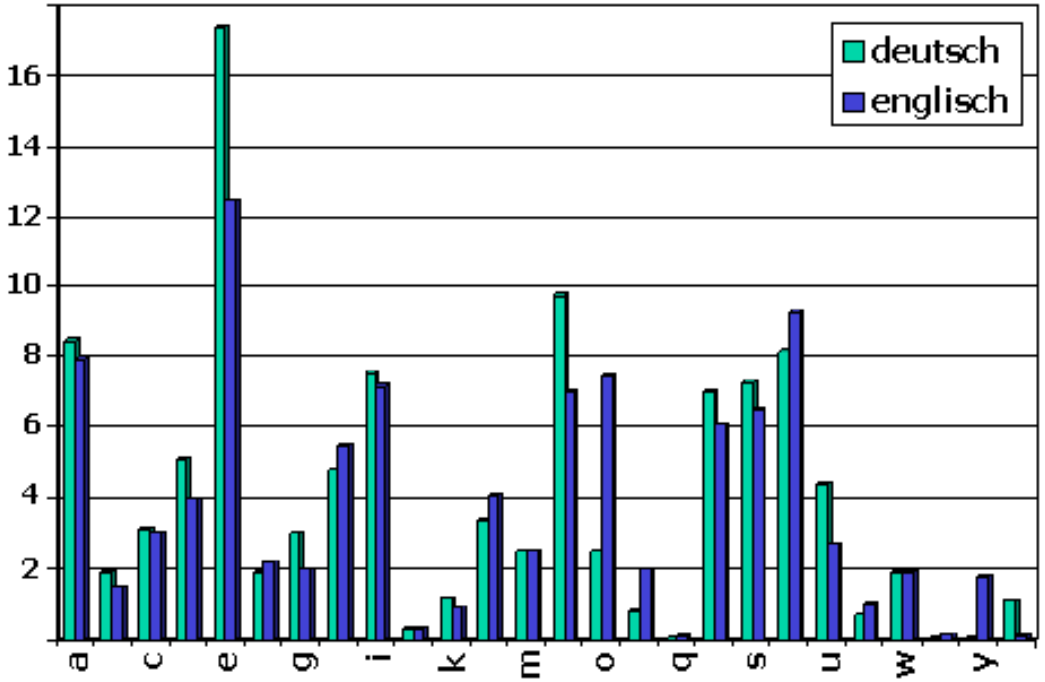
eK is a mono-alphabetic substitution cipher.

Example: ROT13: shift each letter by 13 places.
In Unix: „`tr a-zA-Z n-za-mN-ZA-M`"

Security of such substitution

Huge key space: 26 letters = 26! Keys, but:

- Easily broken by analysing the frequency of letters in texts written in a certain language
- Tables can be used for pairs of letters, or forbidden pairs...
- Computers can break such „encryption“ in real time.



Source: http://home.nordwest.net/hgm/krypto/intro.htm

Improvement: Homophonic Substitutions

Idea:

- Map groups of letters into new groups
- Expand the alphabet (numbers, special characters)

Important:

- Try to find a mapping that results in an equal distribution of characters; this reduces the risks of attacks based on a letter frequency analysis

Problem:

- If an attacker gets access to plain and cipher text of one message: Game Over....
  - N.B.: The plain text is in many cases partly known

Vigenère-Cyphers

- Map each plain text character p into cipher text character c:

  $c = (p + K) \bmod 26$ (K=Key)

- K is not identical for each character, but for instance a „key phrase“:

```
EFFENBERGEFFENBERGEFF  (Key)
VORLESUNGSICHERHEITIN  (plain text)
ZTWPRTYEMWNHLRSLVOXNS  (cipher text)
```

More formally:

A Key is given by sequence of numbers $e = e_1, \ldots, e_t$, where $p_i(a) = (p + e_i) \bmod n$, defines a permutation for $|A| = n$.

Attacking Vigenère

If the length of the key is known, split text accordingly; then:

- Each block uses a Caesar Cipher.

Example: length = 4

- take 1., 5., 9., etc. character and attack with a letter frequency analysis
- Continue with 2., 6., 10., ...

If the length is unknown: try to guess it and attack it as above

- Computers were built for stupid tasks like this...

One-time Pad









Crypto Museum
cryptomuseum.com

One-time Pads

A one-time pad is a stream cipher defined on $A = \{0,1\}$

- A message $x_1x_2...x_n \in M$ is encrypted by a binary string $K = K_1K_2...K_n$

$$eK(x_1x_2...x_n) = (x_1 \oplus K_1)(x_2 \oplus K_2)...(x_n \oplus K_n)$$

and decrypted by:

$$dK(x_1x_2...x_n) = (x_1 \oplus K_1)(x_2 \oplus K_2)...(x_n \oplus K_n)$$

- Every key sequence is equally likely, so is every plaintext!
- Provides unconditional (information theoretic) security, if key isn't reused!
- Problem: Securely exchanging and synchronizing long keys.

One-time pad

Provably secure cryptosystem, invented by Gilbert Vernam in 1918.

One-time Pad = secret key, at least as long as message to protect

Interpret one-time pad as “noise” for the message, encryption and decryption with XOR:

- 0 means “don’t flip”
- 1 means “flip”

Example:

Encryption

Message	0110010
Pad	1011001
Encrypted	1101011

Decryption

Encrypted	1101011
Pad	1011001
Message	0110010

Security of the One-time Pad

The security of the OTP depends on the randomness of the key.

- **It is hard to define randomness precisely**
- **Fundamental properties:**
  - **Unpredictability: The probability of guessing the next bit is not better than ½ (independent of the number of the bits of a sequence observed).**
  - **Equal Distribution: The number of 1's and 0's is asymptotically equal.**
  - **Pseudo-randomness:** Sequence appears random to a computationally bounded adversary.

https://www.mattblaze.org/blog/neinnines/

Matt Blaze's

EXHAUSTIVE SEARCH

Science, Security, Curiosity

A Cryptologic Mystery

18 SEPTEMBER 2020

Did a broken random number generator in Cuba help expose a Russian espionage network?



I picked up the new book *Compromised* last week and was intrigued to discover that it may have shed some light on a small (and rather esoteric) cryptologic and espionage mystery that I've been puzzling over for about 15 years. *Compromised* is primarily a memoir of former FBI counterintelligence agent Peter Strzok's investigation into Russian operations in the lead up to the 2016 presidential election, but this post is not a review of the book or concerned with that aspect of it.

Proof Sketch

The probability of a key bit being 1 or 0 is exactly equal to ½.

- plaintext bits are not balanced. Let the probability of 0 be x, then the probability of 1 is 1-x.

Probability of ciphertext bits:

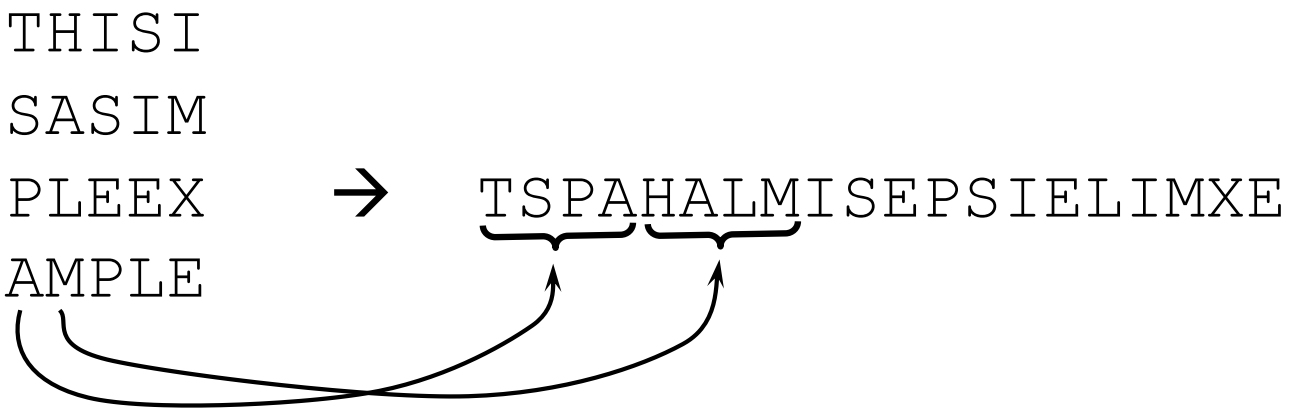
m_i	prob.	k_i	prob.	c_i	prob.
0	x	0	0.5	0	$0.5 \cdot x$
0	x	1	0.5	1	$0.5 \cdot x$
1	$1-x$	0	0.5	1	$0.5 \cdot (1-x)$
1	$1-x$	1	0.5	0	$0.5 \cdot (1-x)$

The probability of a ciphertext bit being 1 or 0 is equal to $0.5 \cdot x + 0.5 \cdot (1-x) = 0.5$.

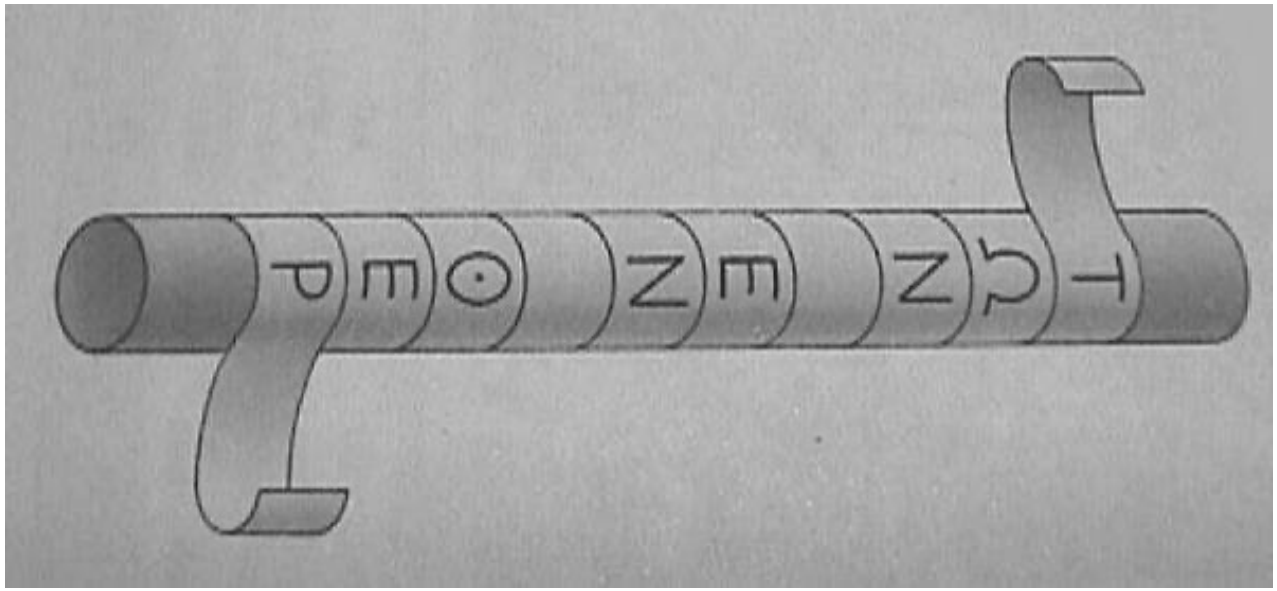
- Ciphertext looks like a random sequence.

Transposition Ciphers

- **Transposition does not change the letters itself, but their position in a text**
- **Example: Write text in groups of 5 letters, and read it in columns**



Historic Version (500 b.C.)



Composite Ciphers

Ciphers based on just substitutions or transpositions are in general not secure

- Ciphers need to be combined.

But:

- Two substitutions are actually only another substitution,
- Two transpositions are actually only one transposition,

A substitution followed by a transposition makes a new and (presumably) more secure cipher.

- Difficult to do by hand → invention of cipher machines



© J. Posegga,

The Enigma

Position determines the encryption key

Rotor

Display for the cipher text

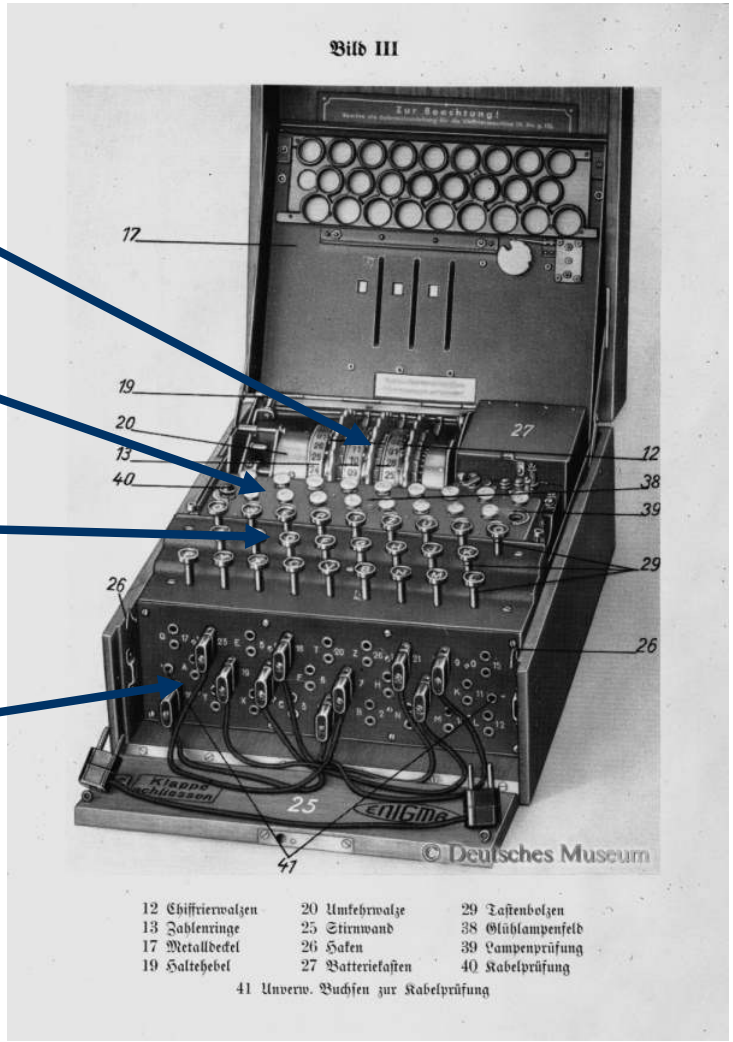
Lamps

keyboard

Input of plain text

wiring

Additional permutations



Source: Deutsches Museum, München

Geheim!
Nicht ins Flugzeug mitnehmen!

OKH-Maschinenschlüssel A Nr. 39

№ 00014

	Datum	Walzenlage			Ringstellung			Steckerverbindungen										Kenngruppen			
0	31.	V	II	IV	17	09	02	KT	AJ	IV	UR	NY	HZ	GD	XF	PB	CQ	sfy	azy	zkq	bqi
0	30.	I	III	V	22	13	10	UE	PL	AY	TB	ZH	WM	OJ	DC	KN	SI	iuy	swz	omo	myj
0	29.	V	IV	II	04	01	25	WJ	VD	PO	MQ	FX	ZR	NE	LG	UC	BK	rui	kao	fqi	rwu
0	28.	II	III	IV	05	03	12	HR	TJ	LD	IO	CN	GX	QK	PZ	WS	AF	ioy	kjv	yko	fpz
0	27.	I	II	III	10	20	15	AQ	ZK	MU	GH	ST	LN	XY	IJ	BF	RV	ggf	jus	lrs	glc
0	26.	II	V	I	15	09	06	DS	UL	ZJ	OI	HN	PT	RK	YC	XQ	GB	orl	rht	ksz	ego
0	25.	V	IV	III	26	07	18	WA	QD	XS	UY	LG	JI	FB	HK	MT	CE	pfr	ijw	zgg	ygj
0	24.	III	I	IV	04	19	24	OH	XM	DJ	IL	VU	KG	QZ	BT	FR	AS	nbt	pvd	eqo	wyn
0	23.	I	IV	V	11	17	01	QJ	GY	SH	OX	ZB	PL	FA	WI	VK	ND	hhv	hhq	kul	hmf
0	22.	IV	I	III	21	11	17	CV	LE	KN	UH	YJ	TI	RB	PZ	FA	MO	jlw	vrh	vya	pbf
0	21.	I	V	II	06	21	10	JN	UX	YT	BG	DR	QC	KE	SP	HZ	LA	zit	jlc	jbl	pvi
0	20.	V	II	III	07	18	04	ZO	NW	SM	VY	XT	UR	OC	LB	AQ	HF	otx	gns	xeg	nvo
0	19.	IV	V	I	08	09	22	IT	YK	BL	RZ	VP	PN	JW	QO	MS	AE	lyx	jua	zju	nss
0	18.	I	IV	III	26	16	11	BU	TS	VH	JL	WX	AY	KG	ZM	PD	NF	ize	ysj	skw	znr
0	17.	III	V	I	11	22	16	GY	JN	SF	KI	LB	QD	UX	CW	HR	MA	xvd	kkb	poi	fug
0	16.	V	I	IV	04	09	24	QL	EY	BG	MN	ZO	AW	TC	VX	FS	HP	afp	uah	tpn	npf
0	15.	II	V	III	03	20	14	JD	BM	XR	LG	PC	OF	ZI	YH	VK	EW	nfk	pvm	vue	cpr
0	14.	IV	I	II	25	12	15	BT	OW	SN	DA	ZL	VP	QX	UE	HR	MC	zgc	omz	pdf	xuq
0	13.	I	V	IV	07	18	05	IW	NB	XO	YS	AJ	MQ	VH	FT	UL	RE	zor	oom	odl	ijs
0	12.	IV	III	II	19	03	21	CN	LG	IZ	DO	SB	VR	TQ	KM	JF	AX	eqk	whq	avc	zpf
0	11.	V	II	I	08	20	14	HV	FP	CM	AJ	OU	YB	WS	NT	GK	EZ	hvm	icd	nxo	yxk
0	10.	IV	V	III	21	08	03	IJ	XR	ZV	NT	CK	OU	EB	FL	MY	HD	bgd	xke	gsg	sgs
0	9.	III	I	II	14	16	06	LN	IK	HS	DB	TX	CG	WY	EV	OF	RA	myh	ncz	xvx	ees
0	8.	IV	III	I	09	18	14	RG	XU	WZ	AP	LF	IY	SQ	DO	VJ	HT	ooq	xeo	ocn	kde
0	7.	II	I	V	18	13	24	EK	RO	JX	WV	HS	QP	BZ	MU	TN	CA	fmc	mkh	lhe	tmq
0	6.	III	II	IV	23	01	17	DC	VG	OL	UA	EK	ZH	YX	PW	IM	RP	tlc	wbj	sre	kjd
0	5.	V	III	I	19	23	15	QP	DG	ZJ	NK	SB	IC	FT	ER	UV	HA	hnp	wla	shv	spd
0	4.	IV	II	V	26	04	03	MX	QO	HI	TB	GA	KP	LZ	CS	WJ	NV	clc	jdh	yoq	hwt
0	3.	V	III	II	01	02	23	BI	DY	PO	SJ	FN	LB	RK	GX	AH	CU	jty	bzy	kdh	asq
0	2.	I	V	III	16	07	02	ZO	IA	VM	CT	PX	YB	HU	SD	RN	EL	uqn	nsx	jqk	pzb
0	1.	IV	I	V	20	05	10	SX	KU	QP	VN	JG	TC	LA	WM	OB	ZF	sro	eej	fnz	szk

© J. Posegga,

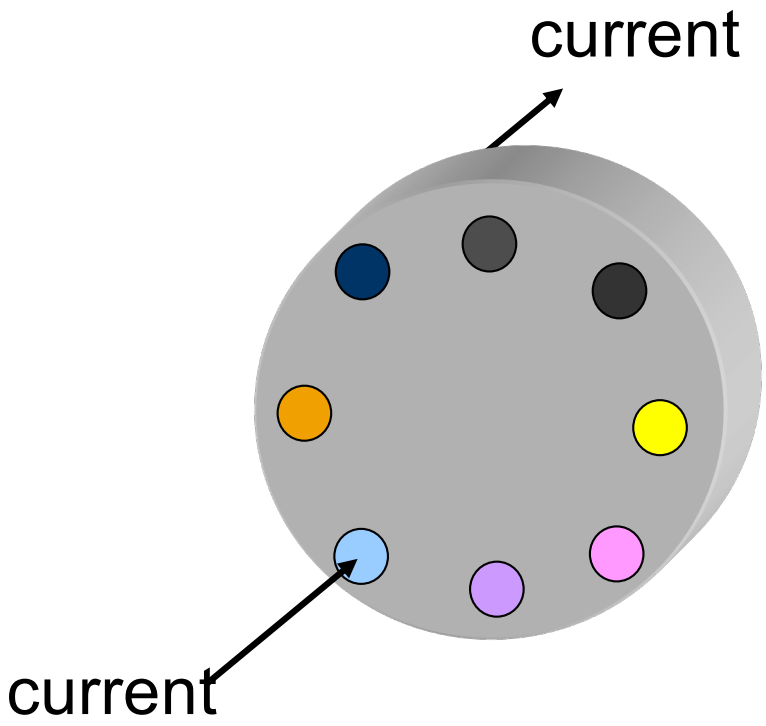
Enigma (I)

Consists of a sequence of rotors.

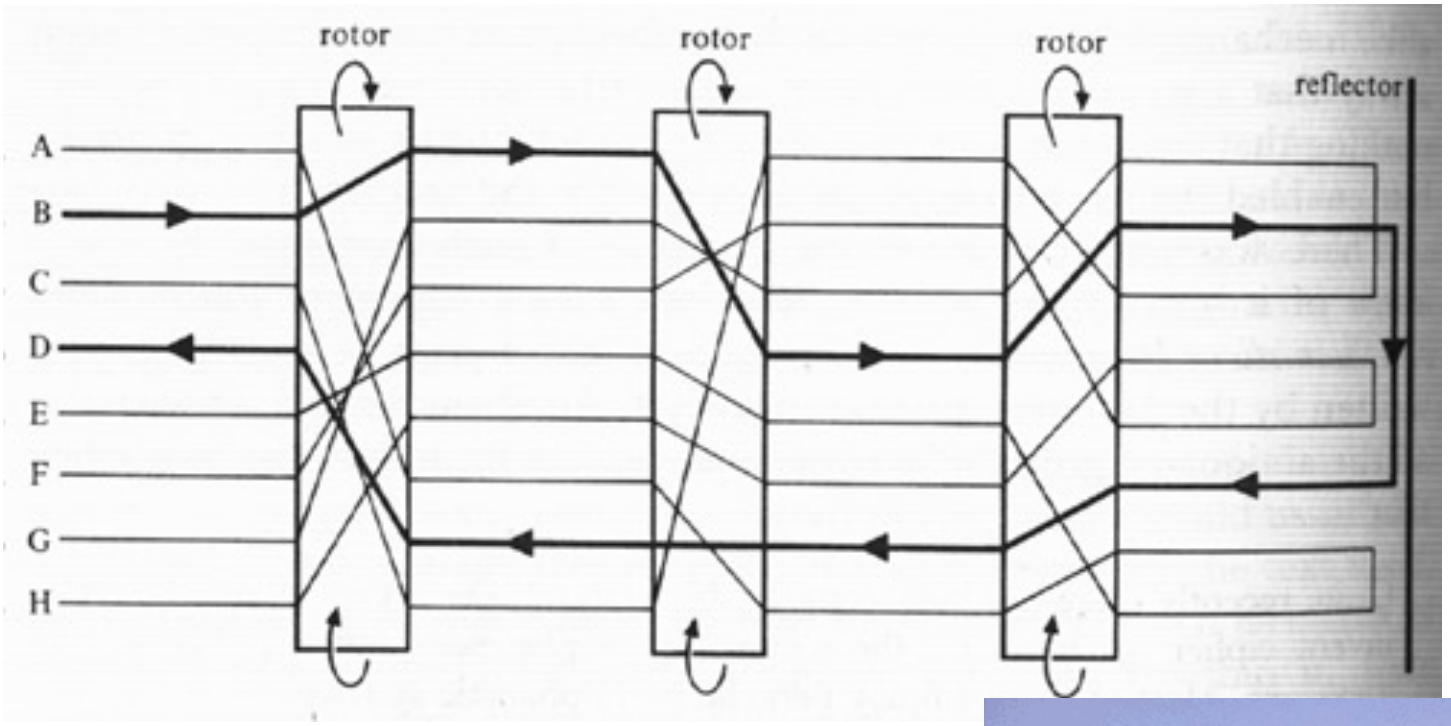
Each rotor is an electrically isolated device with 26 cointacts on each side.

Each input contact is connected to an output contact, the scheme is secret.

-> each rotor defines a substitution.



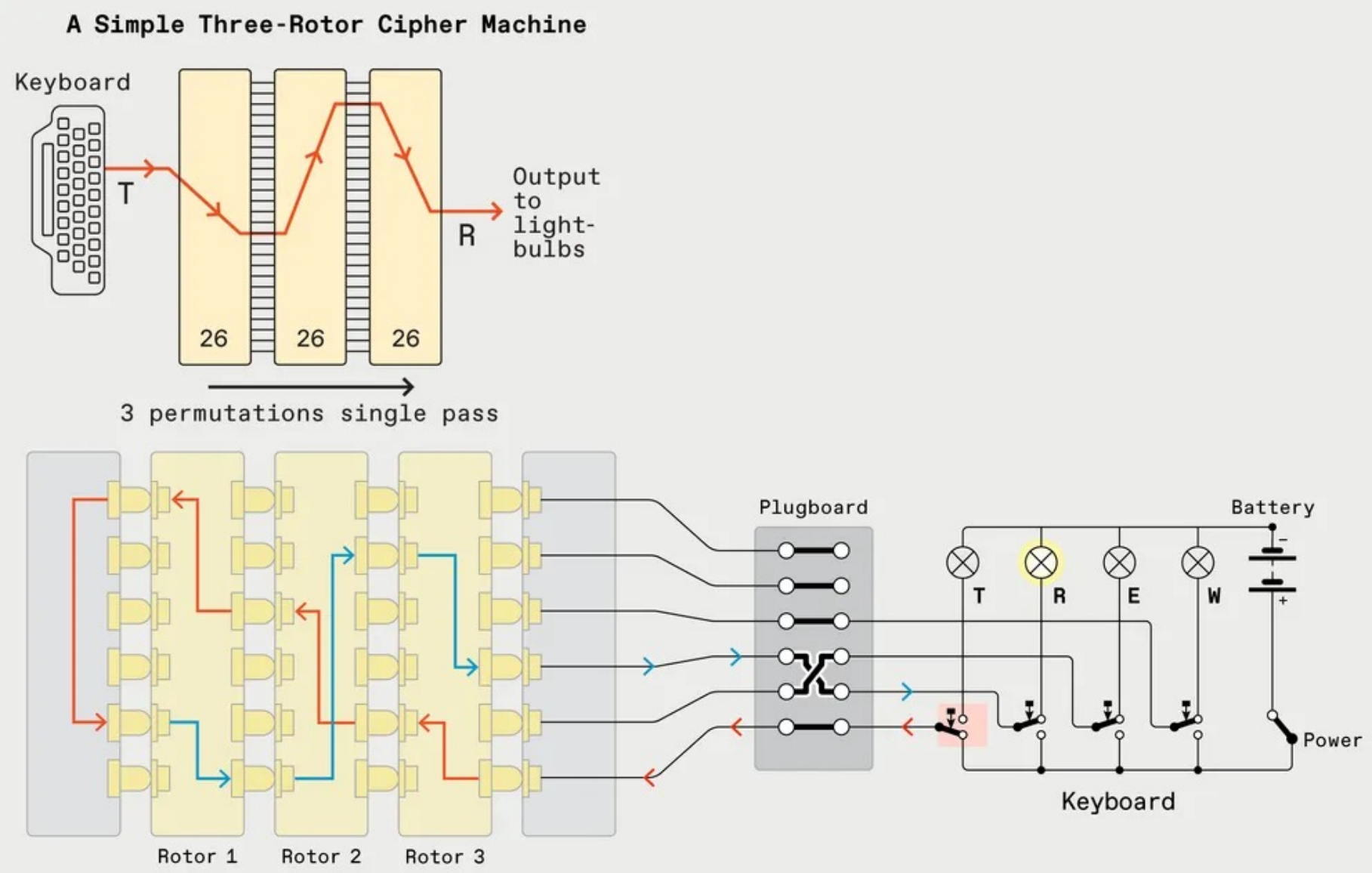
The Enigma Machine: Beyond Caesar Ciphers







(Source: Andrew Hodges: *Alan Turing: The Enigma*; Simon and Schuster; 1983)



https://spectrum.ieee.org/the-scandalous-history-of-the-last-rotor-cipher-machine

Properties of the Enigma

A letter never maps to itself

Encryption and decryption works with identical settings

$2 \cdot 10^{20}$ different keys

Code books were used to transport the keys:

- A distinct setting of rotors for each day to encrypt a message („session“) key
- Individual message keys were used to encrypt transmissions

The Enigma was believed to be very secure

- this was in fact true, compared to the state of the art in encryption at that time
- However, the scheme was broken...

Cyptanalysis of the Enigma

The Enigma scheme was broken during WW II, without that the Germans became aware of it.

- The Enigma was still believed to be secure (even after WW II).

Weaknesses of the Enigma scheme:

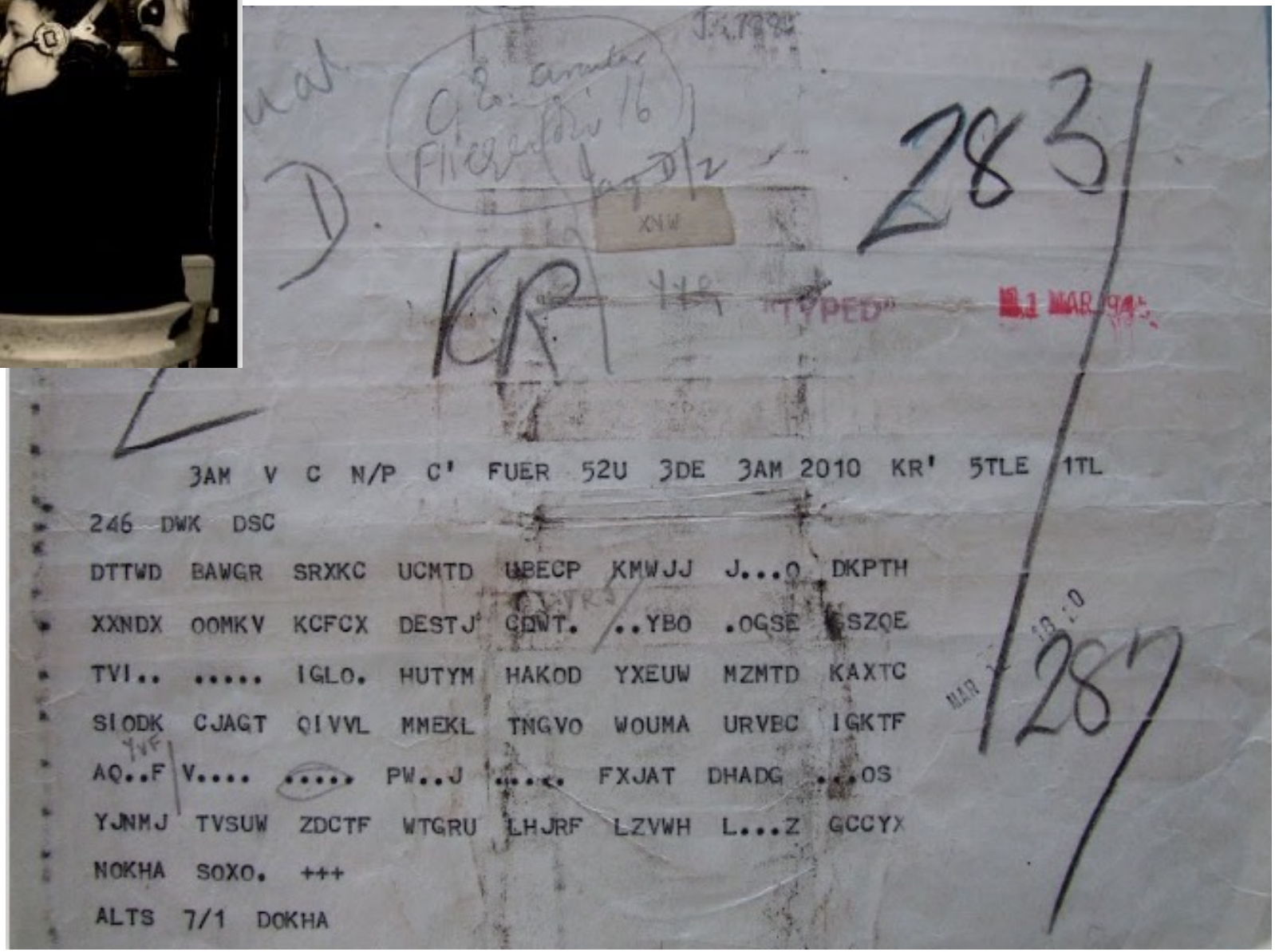
- The machine (or parts of it) became accessible to the adversary
- Certain specifics of the ciphering scheme

plus:

- The adversary spend enormous effort on breaking the scheme (Bletchley Park, Polish Scientists)
- Lots of cipher text was available

Examples:





3AM V C N/P C' FUER 52U 3DE 3AM 2010 KR' 5TLE 1TL

246 DWK DSC

DTTWD BAWGR SRXKC UCMTD UBECP KMWJJ J...O DKPTH

XXNDX OOMKV KCFCX DESTJ CDWT. ..YBO .OGSE SZQE

TVI.. ..... IGLO. HUTYM HAKOD YXEUW MZMTD KAXTC

SIODK CJAGT QIVVL MMEKL TNGVO WOUMA URVBC IGKTF

AQ..F V.... ..... PW..J ..... FXJAT DHADG ...OS

YJNMJ TVSUW ZDCTF WTGRU LHJRF LZVWH L...Z GCCYX

NOKHA SOXO. +++

ALTS 7/1 DOKHA

© J. Posegga,



© J. F

Loss of Machines and Code Books

The Polish secret service received an Enigma in 1927; the machine was mistakenly sent to a Germany company in Poland.

- **Although this was a „civil version“, the principles underlying the Enigma became known to the adversary.**

The British Army attacked poorly armed weather ships of the Germans, which were also using the Enigma

- **The adversary got access to code books (without the knowledge of the Germans)**
- **Many (old) messages could be decrypted**

Specifics of the Enigma Scheme

A letter is never mapped to itself!

-> Known Plaintext Attack:

- **If a certain word of the plain text is known, one can determine the positions in the cipher text, where this word possibly appears.**

Example:

```
GAAHADFDEGFDTGH
WAHRHEIT
 WAHRHEIT
  WAHRHEIT (*)
   WAHRHEIT
    WAHRHEIT (*)
     WAHRHEIT
      WAHRHEIT (*)
       WAHRHEIT (*)
```

„Wahrheit“ is known to appear in plain text.

(*) possible positions in cipher text.

Block ciphers & stream ciphers

Block ciphers: encrypt sequences of "long" data blocks without changing the key

- security relies on design of encryption function
- typical block length: 64 bits, 128 bits

Stream ciphers: encrypt sequences of "short" data blocks under a changing key stream

- security relies on design of key stream generator
- encryption can be quite simple, often XOR
- typical block length: 1 bit, 1 byte, 8-bit word

-> Example: WEP encryption in IEEE802.11b